数据结构：线性表、栈与队列
复习讲义

目录

1 抽象数据类型 (Abstract Data Type) 3
1.1 数据类型与 ADT . . . 3

2 线性表 ADT (The List ADT) 3
2.1 对象与操作 . . . 3
2.2 数组实现 (Simple Array Implementation) . . . 4
2.3 单链表实现 (Linked Lists) . . . 4
2.4 双向循环链表 (Doubly Linked Circular Lists) . . . 5

3 线性表应用 (Applications of Lists) 5
3.1 多项式 ADT . . . 5
3.2 多项式表示 1：数组 . . . 6
3.3 多项式表示 2：链表 . . . 6
3.4 多重链表 (Multilists) . . . 6
3.5 游标实现 (Cursor Implementation) . . . 6

4 栈 ADT (The Stack ADT) 7
4.1 对象与操作 . . . 7
4.2 栈的实现 . . . 8

5 栈的应用 (Applications of Stacks) 8
5.1 平衡符号检查 . . . 8
5.2 后缀表达式求值 . . . 9
5.3 中缀转后缀 . . . 9
5.4 函数调用与系统栈 . . . 10

6 队列 ADT (The Queue ADT) 10
6.1 对象与操作 . . . 10
6.2 数组实现与循环队列 . . . 11

7 全章总结 **11**
7.1 复习主线 11

1 抽象数据类型 (Abstract Data Type)

1.1 数据类型与 ADT

课件定义：

$$\text{Data Type} = \{\text{Objects}\} \cup \{\text{Operations}\}.$$

例如：

$$\texttt{int} = \{0, \pm1, \pm2, \ldots, \texttt{INT_MAX}, \texttt{INT_MIN}\} \cup \{+, -, \times, \div, \%, \ldots\}.$$

抽象数据类型 (ADT) 是一种组织数据类型的方式：对象及其操作的**规格说明**与对象的**表示方式**、操作的**具体实现**分离。

ADT 的复习重点

ADT 关心“能做什么”，实现关心“怎么做”。同一个 ADT 可以有不同实现，例如 List 可以用数组、链表或游标实现。

2 线性表 ADT (The List ADT)

2.1 对象与操作

List 的对象可写为：

$$(item_0, item_1, \ldots, item_{N-1}).$$

课件列出的典型操作包括：

- 求表长 N。
- 打印表中所有元素。
- 置空表。
- 查找第 k 个元素，其中 $0 \le k < N$。
- 在第 k 个元素之后插入新元素。
- 删除某个元素。
- 找当前元素的后继。
- 找当前元素的前驱。

2.2 数组实现 (Simple Array Implementation)

数组实现使用顺序映射：

$$\texttt{array[i]} = item_i.$$

若数组起始地址为 `array`，则 `array+i` 中存放 $item_i$。

操作/性质	时间	说明
Find_Kth	$O(1)$	可直接通过下标访问
插入	$O(N)$	需要移动大量元素
删除	$O(N)$	需要移动大量元素
空间	$O(\text{MaxSize})$	最大容量必须预估

2.3 单链表实现 (Linked Lists)

链表中结点可以分散在内存不同位置，通过指针连接。课件示例中 ZHAO、QIAN、SUN、LI 等结点的物理地址可不同，逻辑顺序由 `next` 决定。

```
typedef struct list_node *list_ptr;
typedef struct list_node {
    char data[4];
    list_ptr next;
};
list_ptr ptr;
```

初始化两个结点并连接：

```
N1 = (list_ptr)malloc(sizeof(struct list_node));
N2 = (list_ptr)malloc(sizeof(struct list_node));
N1->data = "ZHAO";
N2->data = "QIAN";
N1->next = N2;
N2->next = NULL;
ptr = N1;
```

插入

若要在 `node` 后插入 `temp`：

```
temp->next = node->next;
node->next = temp;
```

顺序不能反过来，否则原来的后继结点会丢失。已知插入位置时，插入为 $O(1)$。

删除

若 pre 是待删除结点 node 的前驱：

```
pre->next = node->next;
free(node);
```

已知前驱时，删除为 $O(1)$。删除第一个结点会比较特殊，因此常加**哑头结点 (dummy head node)** 统一操作。

2.4 双向循环链表 (Doubly Linked Circular Lists)

单链表找前驱需要从头扫描，因此引入双向链表。典型结点为：

```
typedef struct node *node_ptr;
typedef struct node {
    node_ptr llink;
    element item;
    node_ptr rlink;
};
```

双向链表应满足：

$$\texttt{ptr == ptr->llink->rlink}, \quad \texttt{ptr == ptr->rlink->llink}.$$

课件使用带头结点的双向循环链表：空表中头结点的左右指针都指向自己。这样查找前驱、删除当前结点等操作更方便。

3 线性表应用 (Applications of Lists)

3.1 多项式 ADT

多项式对象可表示为：

$$P(x) = a_1 x^{e_1} + a_2 x^{e_2} + \cdots + a_n x^{e_n},$$

也可看作有序二元组集合 $\langle e_i, a_i \rangle$，其中 a_i 是系数，e_i 是非负整数指数。

典型操作：

- 求次数 $\max\{e_i\}$。
- 多项式加法。
- 多项式减法。
- 多项式乘法。
- 多项式求导。

3.2 多项式表示 1：数组

```
typedef struct {
    int CoeffArray[ MaxDegree + 1 ];
    int HighPower;
} *Polynomial;
```

数组下标表示指数，数组元素表示系数。优点是实现简单；缺点是对高次数稀疏多项式浪费空间。例如 $10x^{1000} + 5x^{14} + 1$ 这种多项式中，大多数系数为 0。

3.3 多项式表示 2：链表

每一项用一个结点表示：

```
typedef struct poly_node *poly_ptr;
struct poly_node {
    int Coefficient;
    int Exponent;
    poly_ptr Next;
};
typedef poly_ptr Polynomial;
```

结点按指数排序。链表表示适合稀疏多项式，插入、合并同类项和多项式加法更自然。

3.4 多重链表 (Multilists)

课件例子：有 40000 名学生和 2500 门课程，需要：

- 对每门课打印学生名单。
- 对每个学生打印所选课程列表。

若用二维数组 `Array[40000][2500]`，空间极大且多数位置为空。多重链表把一个选课记录同时挂在“学生链”和“课程链”上，一个结点服务两种遍历需求。课件还提示自学稀疏矩阵表示。

3.5 游标实现 (Cursor Implementation)

游标实现用于没有指针的语言中模拟链表。核心特征：

1. 数据存储在结构数组中，每个结构包含数据和指向下一个结构的“游标”。
2. 新结点从全局数组的空闲链表中分配，释放时归还空闲链表。

典型结构：

下标	Element	Next
0	空闲链表头	指向第一个空闲位置
1..S-1	数据或空闲单元	下一结点游标

游标版 `malloc`：

```
p = CursorSpace[0].Next;
CursorSpace[0].Next = CursorSpace[p].Next;
```

游标版 `free(p)`：

```
CursorSpace[p].Next = CursorSpace[0].Next;
CursorSpace[0].Next = p;
```

课件说明：游标实现接口与指针实现相同，而且通常因缺少系统内存管理调用而更快。

4 栈 ADT (The Stack ADT)

4.1 对象与操作

栈 (Stack) 是后进先出 (LIFO) 的线性表，插入和删除都只在栈顶进行。
对象：含零个或多个元素的有限有序表。
典型操作：

- `IsEmpty(Stack S)`
- `CreateStack()`
- `DisposeStack(Stack S)`
- `MakeEmpty(Stack S)`
- `Push(ElementType X, Stack S)`
- `Top(Stack S)`
- `Pop(Stack S)`

栈错误类型

对空栈执行 `Pop` 或 `Top` 是 ADT 错误；对满栈执行 `Push` 是实现错误，而不是 ADT 本身的逻辑错误。

4.2 栈的实现

链表实现

常用带头结点的链表实现。栈顶是头结点之后的第一个结点。

- `Push`：新结点先指向当前首结点，再让头结点指向新结点。
- `Top`：返回 `S->Next->Element`。
- `Pop`：保存首结点，头结点跳过它，再释放该结点。

课件提醒：频繁 `malloc` 和 `free` 代价较高，可维护一个回收栈作为 recycle bin。

数组实现

```
struct StackRecord {
    int Capacity;
    int TopOfStack;
    ElementType *Array;
};
```

约定：`TopOfStack == -1` 表示空栈；`Push` 时自增，`Pop` 时自减。

课件强调：

- 栈模型必须封装，除栈例程外，其他代码不应直接访问 `Array` 或 `TopOfStack`。
- `Push`、`Pop`、`Top` 前必须做错误检查。

5 栈的应用 (Applications of Stacks)

5.1 平衡符号检查

检查括号 `()`、方括号 `[]` 和花括号 `{}` 是否匹配。

```
Make an empty stack S;
while ( read a character c ) {
    if ( c is an opening symbol )
        Push(c, S);
    else if ( c is a closing symbol ) {
        if ( S is empty ) ERROR;
        else if ( Top(S) does not match c ) ERROR;
        else Pop(S);
    }
}
if ( S is not empty ) ERROR;
```

若表达式长度为 N，时间复杂度为 $O(N)$。这是在线算法。

5.2 后缀表达式求值

后缀表达式也叫逆波兰记法 (Reverse Polish Notation)。例如：

- 中缀：`a+b*c-d/e`
- 前缀：`- + a * b c / d e`
- 后缀：`a b c * + d e / -`

后缀求值规则：

1. 遇到操作数，压栈。
2. 遇到操作符，弹出所需操作数，计算后把结果压栈。
3. 扫描结束后，栈顶就是结果。

课件例子：

$$\texttt{6 2 / 3 - 4 2 * +} = 8.$$

计算过程为：$6/2=3$，$3-3=0$，$4*2=8$，$0+8=8$。时间复杂度为 $O(N)$，且不需要知道优先级规则。

5.3 中缀转后缀

基本观察：

- 操作数在中缀和后缀中的相对顺序不变。
- 高优先级操作符应先输出。
- 操作符用栈暂存，遇到更低优先级的操作符时，弹出栈内高优先级操作符。

例子：

$$\texttt{a + b * c - d} \quad \Rightarrow \quad \texttt{a b c * + d -}.$$

含括号例子：

$$\texttt{a * (b + c) / d} \quad \Rightarrow \quad \texttt{a b c + * d /}.$$

括号与结合性

- 除处理右括号时，永远不要弹出左括号。
- 左括号作为 incoming symbol 时优先级最高，进入栈后优先级最低。
- 要区分 in-stack precedence 与 incoming precedence。
- 减法左结合：`a-b-c` 转为 `a b - c -`。
- 指数运算右结合：`2^2^3` 应转为 `2 2 3 ^ ^`，而不是 `2 2 ^ 3 ^`。

5.4 函数调用与系统栈

函数调用使用系统栈保存栈帧 (stack frame)，其中包含：

- 旧帧指针 (old frame pointer)。
- 返回地址 (return address)。
- 局部变量 (local variables)。

递归程序通常更简洁，但非递归版本通常更快。尾递归可被编译器优化为循环。课件示例：

```
void PrintList( List L )
{
    if ( L != NULL ) {
        PrintElement( L->Element );
        PrintList( L->next );
    }
}
```

这是尾递归，可改为循环遍历链表。课件提醒：若列表有一百万个元素，递归可能导致系统栈溢出。

6 队列 ADT (The Queue ADT)

6.1 对象与操作

队列 (Queue) 是先进先出 (FIFO) 的线性表：插入在一端，删除在另一端。
典型操作：

- `IsEmpty(Queue Q)`
- `CreateQueue()`
- `DisposeQueue(Queue Q)`
- `MakeEmpty(Queue Q)`
- `Enqueue(ElementType X, Queue Q)`
- `Front(Queue Q)`
- `Dequeue(Queue Q)`

6.2 数组实现与循环队列

链表实现队列很直接；课件重点讲数组实现：

```
struct QueueRecord {
    int Capacity;
    int Front;
    int Rear;
    int Size;          /* optional */
    ElementType *Array;
};
```

在普通数组中，反复出队会导致前面位置空出来但无法继续利用。循环队列用取模让下标回绕：

$$\texttt{Rear = (Rear + 1) \% Capacity.}$$

循环队列的满/空判定

若只用 `Front` 和 `Rear`，常保留一个空格区分“满”和“空”。若额外维护 `Size` 字段，就可以避免浪费这个空格。也可以使用标志位等其他方法。

课件用操作系统作业调度举例：作业按到达顺序入队，CPU 或调度器按先到先服务顺序出队。

7 全章总结

7.1 复习主线

1. ADT 把规格与实现分离。
2. 数组表支持 $O(1)$ 随机访问，但插入删除通常为 $O(N)$。
3. 链表插入删除局部操作为 $O(1)$，但查找位置通常需要遍历。
4. 双向循环链表适合需要频繁找前驱或删除当前结点的场景。
5. 栈是 LIFO，核心应用包括括号匹配、后缀求值、中缀转后缀和函数调用。
6. 队列是 FIFO，数组实现时要掌握循环队列及满/空判定。

“线性结构的关键，是理解访问位置、插入删除和内存表示之间的取舍。”